Przetwarzanie języka naturalnego

Sprawozdanie z laboratoriów 1–4

Analiza korpusu języka serbskiego: statystyka liter, entropia i kodowanie Huffmana, prawo Zipfa, graf sąsiedztwa wyrazów, sieć Kohonena

Autor	Przemysław Kondrat

Język korpusu	serbski (cyrylica, 30 liter)
Źródło danych	serbska Wikipedia (sr.wikipedia.org), API MediaWiki
Język programowania	Python 3 (numpy, scipy, matplotlib, networkx, stanza, reportlab)
Data	05.11.2026

Spis treści

Wprowadzenie **3**

Struktura programu . . . 3

Słowniczek pojęć . . . 4

1. Laboratorium 1 – wybór języka i przygotowanie korpusu **5**

1.1. Jak to zrobiliśmy . . . 5

1.2. Wynik czyszczenia . . . 5

2. Laboratorium 2 – statystyka liter, entropia i kodowanie Huffmana **7**

2.1. Jak to zrobiliśmy . . . 7

2.2. Udział liter – prawdopodobieństwa . . . 7

2.3. Entropia i kod Huffmana – jak działają (na małym przykładzie) . . . 8

2.4. Wyniki dla całego korpusu . . . 9

3. Laboratorium 3 – pokrycie tekstu, prawo Zipfa, graf sąsiedztwa **11**

3.1. Wyrazy pokrywające 90% tekstu . . . 11

3.2. Prawo Zipfa . . . 12

3.3. Graf wyrazów sąsiadujących i jego rdzeń . . . 13

4. Laboratorium 4 – sieć Kohonena: rzeczowniki w przestrzeni czasowników przechodnich **17**

4.1. Analiza składniowa korpusu . . . 17

4.2. Wybór 100 czasowników przechodnich i 50 rzeczowników . . . 17

4.3. Zbiór treningowy – wektory o 100 składowych . . . 19

4.4. Sieć Kohonena – jak działa i jak ją nauczyliśmy . . . 20

4.5. Wynik – mapa rzeczowników . . . 21

4.6. Gdzie są czasowniki w przestrzeni 100D . . . 23

4.7. Czy wynik zależy od wyboru czasowników? . . . 24

5. Sprawdzenie poprawności wyników **27**

6. Podsumowanie **28**

Realizacja wymagań . . . 28

Najważniejsze wnioski . . . 28

Uruchomienie programu . . . 28

Bibliografia . . . 28

Wprowadzenie

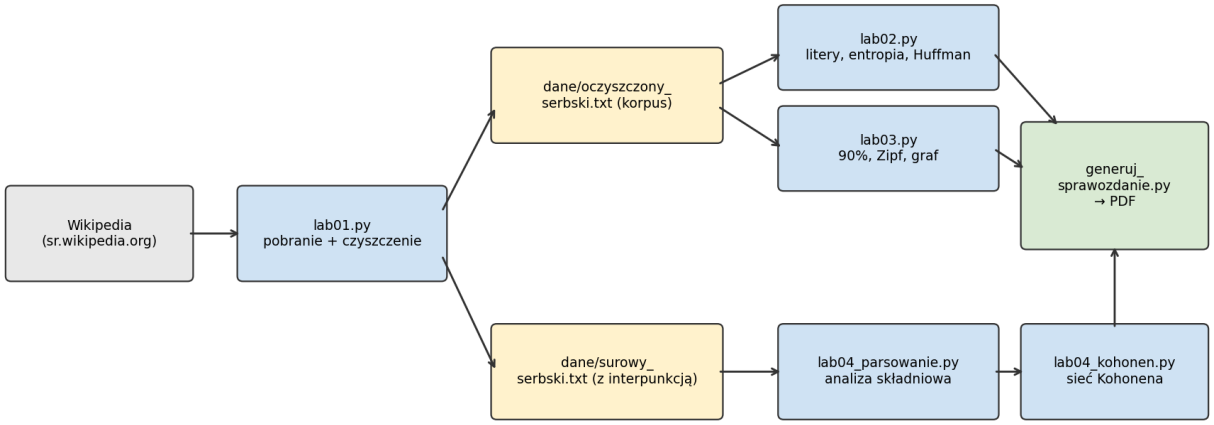
Celem laboratoriów było przeanalizowanie dużego tekstu w wybranym języku metodami przetwarzania języka naturalnego: od statystyki pojedynczych liter, przez statystykę wyrazów i ich sąsiedztwa, aż po sieć neuronową Kohonena, która grupuje rzeczowniki według czasowników, z którymi się łączą.

Wybrany język: serbski. Serbski zapisuje się m.in. cyrylicą, która ma dokładnie 30 liter, a każda litera odpowiada jednej głosce (zasada „piši kao što govoriš” – pisz tak, jak mówisz). Dzięki temu łatwo sprawdzić, że w korpusie zostały wyłącznie litery alfabetu. Bogata odmiana wyrazów przez przypadki jest z kolei ciekawa przy badaniu prawa Zipfa i wymaga analizy składniowej w laboratorium 4.

Dane: 20 artykułów z serbskiej Wikipedii (ok. 1 mln znaków), głównie o historii i geografii Serbii i regionu.

Struktura programu

Każde laboratorium to osobny skrypt w Pythonie. Wspólne ustawienia (alfabet, ścieżki do plików) są w `wspolne.py`, a `uruchom_wszystko.py` uruchamia wszystkie etapy po kolei. Skrypt `sprawdz_wyniki.py` niezależnie przelicza wyniki i potwierdza ich poprawność (rozdział 5). Kod i wyniki są w repozytorium GitHub; opis uruchomienia jest w pliku README.md.



Rys. 1. Schemat działania programu – przepływ danych między skryptami.

Słowniczek pojęć

Pojęcie	Wyjaśnienie
korpus	duży zbiór tekstu przygotowany do analizy; u nas: same wyrazy z liter serbskiego alfabetu
token / typ	token to każde wystąpienie wyrazu w tekście, typ to różny wyraz (w zdaniu „и ја и ти” są 4 tokeny i 3 typy)
lemat	forma podstawowa wyrazu, np. цркву, цркве, црквом → црква (cerkiew)
prawdopodobieństwo p	udział litery w tekście: p = liczba wystąpień / liczba wszystkich liter
entropia H	średnia liczba bitów na znak, poniżej której nie da się zakodować tekstu (miara „nieprzewidywalności” tekstu)
kod Huffmana	kod, w którym częste znaki dostają krótkie ciągi bitów, a rzadkie długie; najkrótszy możliwy kod przypisujący każdemu znakowi stały ciąg bitów
kod prefiksowy	żaden kod nie jest początkiem innego, więc tekst da się odczytać bez separatorów
prawo Zipfa	częstość wyrazu jest odwrotnie proporcjonalna do jego miejsca w rankingu: 2. wyraz występuje ok. 2 razy rzadziej niż 1., 3. ok. 3 razy rzadziej itd.
hapax legomenon	wyraz, który występuje w tekście tylko raz
graf, węzeł, krawędź	węzły to wyrazy, krawędź łączy dwa wyrazy stojące w tekście obok siebie
stopień węzła	liczba krawędzi wychodzących z węzła, czyli liczba różnych sąsiadów wyrazu
rdzeń grafu	100 węzłów o największej liczbie sąsiadów
czasownik przechodni	czasownik, który łączy się z dopełnieniem w bierniku: „podjąć *decyzję*”, „zdobyć *miasto*”
kolokacja	częste połączenie wyrazów, u nas: czasownik + jego dopełnienie
wektor cech	opis obiektu listą liczb; u nas rzeczownik = 100 liczb (po jednej na czasownik)
sieć Kohonena (SOM)	sieć neuronowa ucząca się bez nauczyciela, która układa podobne obiekty blisko siebie na dwuwymiarowej mapie
neuron zwycięski (BMU)	neuron sieci najbardziej podobny do danego wektora wejściowego

Tabela 1. Najważniejsze pojęcia używane w sprawozdaniu.

1. Laboratorium 1 – wybór języka i przygotowanie korpusu

Zadanie: wybrać język, pobrać duży tekst w tym języku i zostawić z niego sam korpus wyrazów: bez znaków specjalnych, interpunkcji, liczb i znaków spoza alfabetu.

> **W prostych słowach**
> Komputer ma liczyć litery i wyrazy, więc tekst musi być „czysty”. Gdyby zostały w nim kropki, cyfry albo angielskie słowa, zafałszowałyby statystyki: kropka liczyłaby się jako litera, a „1389” jako wyraz. Dlatego zostawiamy tylko 30 małych liter serbskiej cyrylicy oddzielonych pojedynczymi spacjami.

1.1. Jak to zrobiliśmy

1. **Pobranie tekstu.** Skrypt `lab01.py` pobiera artykuły z serbskiej Wikipedii przez oficjalne API (sam tekst, bez kodu strony), aż zbierze ok. 1 000 000 znaków. Pomija artykuły krótsze niż 5000 znaków (szablonowe zalążki) i duplikaty, a między zapytaniami czeka 1,5 s, żeby nie obciążać serwera.
2. **Zamiana na małe litery** – „Србија” i „србија” to ten sam wyraz.
3. **Usunięcie znaków spoza alfabetu** – każdy znak, który nie jest jedną z 30 liter (`абвгдђежзијклљмнњопрстћуфхцчџш`), zamieniamy na spację. Jedno wyrażenie regularne `[^абв…ш]` usuwa naraz cyfry, interpunkcję, symbole, łacinkę i grekę.
4. **Zwinięcie spacji** – kilka spacji lub nowych linii z rzędu zamieniamy na jedną spację.
5. **Kontrola** – skrypt liczy, ile znaków każdego rodzaju usunął, i sprawdza, czy w wyniku zostały tylko litery i spacje.

Pobrane artykuły (20): Србија, Историја Србије, Београд, Нови Сад, Ниш, Крагујевац, Војводина, Косово и Метохија, Црна Гора, Босна и Херцеговина, Хрватска, Југославија, Социјалистичка Федеративна Република Југославија, Краљевина Југославија, Краљевина Србија, Српско царство, Немањићи, Стефан Немања, Свети Сава, Стефан Душан.

1.2. Wynik czyszczenia

Grupa usuniętych znaków	Liczba	Przykłady (od najczęstszych)
białe znaki (spacje, nowe linie)	159 836	↵ ␣ \u2009 \u2003
interpunkcja (. , ( ) – „ ” % …)	29 766	, .) (- — : „ ” % ; “
cyfry i liczby (np. 1, 2, ²)	21 112	1 0 9 2 3 8 4 5 7 6 ²
litery łacińskie	4 007	i a m e n k r s o t v c
symbole (= + ° € …)	3 921	= ° + € − | >
litery greckie	190	α σ ρ ο ι ν β ς κ ί τ ε
cyrylica spoza alfabetu serbskiego	14	ь ъ ѣ й ї ю
litery innych alfabetów	6	ن ی ش

Tabela 2. Znaki usunięte z tekstu surowego. Białe znaki nie znikają: zamieniane są na pojedynczą spację oddzielającą wyrazy.

Wielkość	Wartość
Liczba znaków tekstu surowego	1 036 853
Liczba znaków po oczyszczeniu	967 373
Wielkie litery zamienione na małe	34 154
Liczba wyrazów (tokenów)	149 373
Liczba różnych wyrazów (typów)	25 608
Różne znaki w korpusie	31 (30 liter + spacja)

Tabela 3. Korpus przed i po oczyszczeniu.

Przykład – zdanie z tekstu surowego i to samo zdanie po oczyszczeniu:

```
Према истом извору, Вишеслав је потицао из лозе неименованог српског кнеза који је за време владавне
цара Ираклија (610–641) довео свој народ у Далмацију и околне области, зачетник је лозе Вишеславића.

према истом извору вишеслав је потицао из лозе неименованог српског кнеза који је за време владавне
цара ираклија довео свој народ у далмацију и околне области зачетник је лозе вишеславића
```

Wniosek: w korpusie zostały wyłącznie wyrazy złożone z liter serbskiego alfabetu. Potwierdza to zarówno kontrola w lab01.py, jak i niezależne testy z rozdziału 5. Korpus zapisano w dane/oczyszczony_serbski.txt, a tekst surowy w dane/surowy_serbski.txt (potrzebny w lab 4, bo do analizy składniowej niezbędne są granice zdań).

2. Laboratorium 2 – statystyka liter, entropia i kodowanie Huffmana

Zadanie: upewnić się, że został sam korpus; w tabeli podać udział procentowy każdej litery (prawdopodobieństwa); policzyć entropię tekstu; wyznaczyć ciągi kodowe Huffmana i porównać je z entropią.

W prostych słowach

Komputer zapisuje każdą literę jako ciąg zer i jedynek (bitów). Gdyby każda z 30 liter miała kod tej samej długości, potrzebowalibyśmy 5 bitów na literę ($2^5 = 32 \geq 30$). Litery nie są jednak równie częste: „a” pojawia się ponad 600 razy częściej niż „ụ”. **Kod Huffmana** wykorzystuje to i daje częstym literom krótkie kody, a rzadkim długie, więc średnio wychodzi mniej bitów. **Entropia** mówi, ile bitów na literę wystarczy w najlepszym możliwym przypadku. Porównujemy więc, jak blisko tego ideału jest Huffman.

2.1. Jak to zrobiliśmy

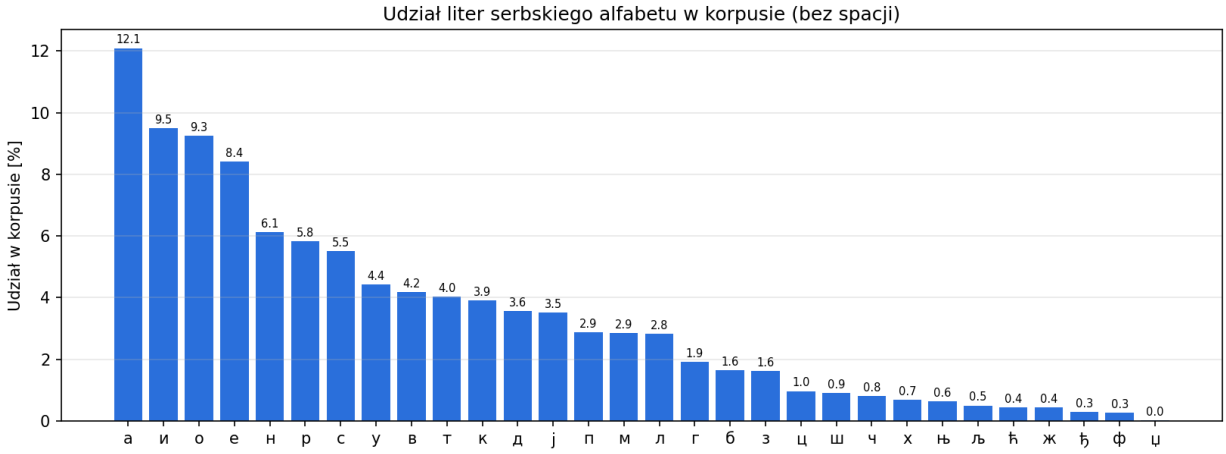
1. Skrypt `lab02_entropia_huffman.py` wczytuje korpus i jeszcze raz sprawdza, że nie ma w nim żadnego znaku spoza alfabetu i spacji. Wynik: brak obcych znaków, występują wszystkie 30 liter.
2. Liczy wystąpienia każdej litery (słownik *Counter*) i dzieli przez liczbę wszystkich liter. Tak powstaje prawdopodobieństwo p każdej litery.
3. Liczy entropię ze wzoru Shannona.
4. Buduje kod Huffmana algorytmem z kolejką priorytetową (opis i przykład w p. 2.3).
5. Koduje cały korpus, dekoduje go z powrotem i sprawdza, czy wyszedł identyczny tekst.
6. Wszystko liczy w dwóch wariantach: same litery (30 symboli) oraz litery + spacja (31 symboli).

2.2. Udział liter – prawdopodobieństwa

Korpus bez spacji zawiera 818 001 liter. Tabela podaje dla każdej litery liczbę wystąpień, udział procentowy i prawdopodobieństwo p, a także „idealną” długość kodu $-\log_2 p$ i faktyczny kod Huffmana (omówione w p. 2.3–2.4).

Lp	Litera	Liczność	Udział [%]	p	$-\log_2 p$	Kod Huffmana	Dł. kodu
1	**а**	98 931	12,094	0,12094	3,048	100	3
2	**и**	77 660	9,494	0,09494	3,397	001	3
3	**о**	75 716	9,256	0,09256	3,433	000	3
4	**е**	68 784	8,409	0,08409	3,572	1110	4
5	**н**	50 077	6,122	0,06122	4,030	0111	4
6	**р**	47 619	5,821	0,05821	4,102	0110	4
7	**с**	45 048	5,507	0,05507	4,183	0100	4
8	**у**	36 186	4,424	0,04424	4,499	11111	5
9	**в**	34 121	4,171	0,04171	4,583	11011	5
10	**т**	32 984	4,032	0,04032	4,632	11010	5
11	**к**	32 069	3,920	0,03920	4,673	11001	5
12	**д**	29 088	3,556	0,03556	4,814	11000	5
13	**ј**	28 824	3,524	0,03524	4,827	10110	5
14	**п**	23 457	2,868	0,02868	5,124	10100	5
15	**м**	23 324	2,851	0,02851	5,132	01011	5
16	**л**	23 043	2,817	0,02817	5,150	01010	5
17	**г**	15 722	1,922	0,01922	5,701	111100	6
18	**б**	13 411	1,639	0,01639	5,931	101011	6
19	**з**	13 235	1,618	0,01618	5,950	101010	6
20	**ц**	7 919	0,968	0,00968	6,691	1011111	7
21	**ш**	7 358	0,900	0,00900	6,797	1011110	7
22	**ч**	6 527	0,798	0,00798	6,970	1011100	7
23	**х**	5 682	0,695	0,00695	7,170	11110111	8
24	**њ**	5 273	0,645	0,00645	7,277	11110110	8
25	**љ**	4 104	0,502	0,00502	7,639	11110100	8
26	**ћ**	3 657	0,447	0,00447	7,805	10111011	8
27	**ж**	3 517	0,430	0,00430	7,862	10111010	8
28	**ђ**	2 367	0,289	0,00289	8,433	111101011	9
29	**ф**	2 155	0,263	0,00263	8,568	1111010101	10
30	**џ**	143	0,017	0,00017	12,482	1111010100	10
	Suma	818 001	100,000	1,00000			

Tabela 4. Udział liter w korpusie (bez spacji) i ich kody Huffmana.



Rys. 2. Udział procentowy liter w korpusie.

Rozkład jest bardzo nierównomierny: cztery samogłoski а, и, о, е to razem ok. 40% liter, a najrzadsze litery (ђ, ф, џ) mają poniżej 0,3% każda. Te różnice wykorzystuje kod Huffmana.

2.3. Entropia i kod Huffmana – jak działają (na małym przykładzie)

Entropia (C. Shannon, 1948) to średnia liczba bitów potrzebna na jeden znak:

$$H = -\Sigma\, p_i \cdot \log_2 p_i$$

Składnik $-\log_2 p_i$ to „idealna” długość kodu znaku i: im rzadszy znak, tym więcej niesie informacji i tym dłuższy może być jego kod. Entropia jest średnią z tych długości, ważoną częstością znaków.

Algorytm Huffmana (D. Huffman, 1952): (1) każdy znak to osobny węzeł z wagą równą liczbie wystąpień; (2) bierzemy dwa węzły o najmniejszych wagach i łączymy je w nowy węzeł o sumie wag; (3) powtarzamy, aż zostanie jeden węzeł (korzeń drzewa). Kod znaku to droga od korzenia do znaku: krok w lewo = 0, w prawo = 1. Częste znaki są łączone na końcu, więc leżą blisko korzenia i mają krótkie kody.

Przykład dla słowa **„абракадабра”** (11 liter): а × 5, б × 2, р × 2, д × 1, к × 1.

Krok	Łączymy węzeł	waga	z węzłem	waga	Nowy węzeł (suma)
1	{д}	1	{к}	1	2
2	{б}	2	{р}	2	4
3	{д, к}	2	{б, р}	4	6
4	{а}	5	{д, к, б, р}	6	11

Tabela 5. Kolejne kroki algorytmu Huffmana dla słowa „абракадабра”.

Litera	Liczność	p	$-\log_2 p$ (ideał)	Kod Huffmana	Długość
а	5	0,455	1,138	0	1
б	2	0,182	2,459	110	3
р	2	0,182	2,459	111	3
д	1	0,091	3,459	100	3
к	1	0,091	3,459	101	3

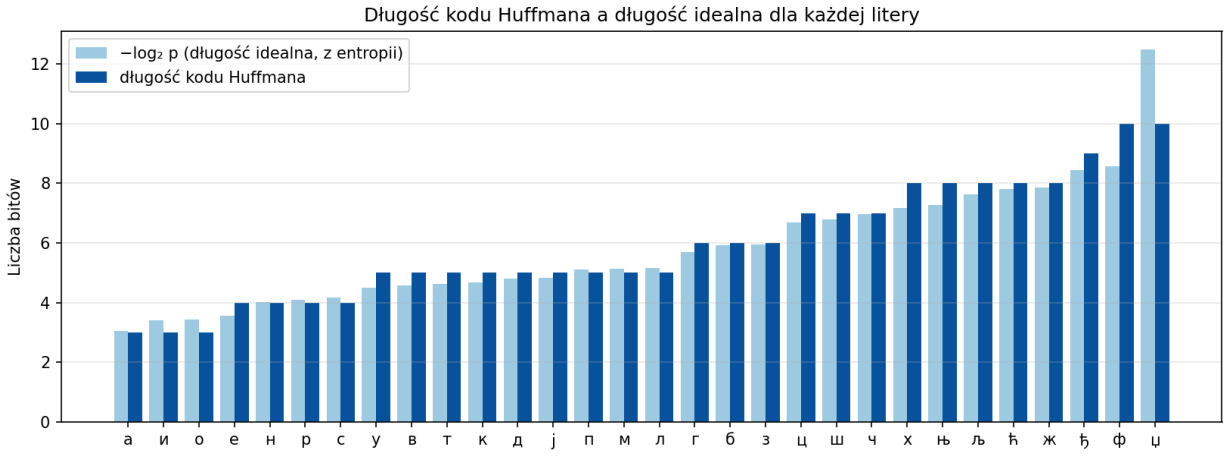
Tabela 6. Kody Huffmana dla słowa „абракадабра”.

Dla tego słowa entropia wynosi H = 2,040 bit/znak, a średnia długość kodu Huffmana L = 2,091 bit/znak. Kod o stałej długości potrzebowałby 3 bitów na znak (33 bitów na całe słowo), a Huffman potrzebuje 23 bitów.

2.4. Wyniki dla całego korpusu

Wariant	n	H [bit]	$H_{max} = \log_2 n$	Redund.	L Huffmana [bit]	L − H	H / L	Kod stały
same litery	30	4,3024	4,9069	12,32%	4,3368	0,0344	99,21%	5 bit
litery + spacja	31	4,2588	4,9542	14,04%	4,2889	0,0301	99,30%	5 bit

Tabela 7. Entropia a średnia długość kodu Huffmana. H_{max} – entropia, gdyby wszystkie znaki były równie częste; redundancja = $1 - H/H_{max}$.



Rys. 3. Długość kodu Huffmana każdej litery w porównaniu z długością idealną $-\log_2 p$.

Wnioski.

- Średnia długość kodu Huffmana (4,3368 bit) jest tylko o 0,0344 bitu większa od entropii (4,3024 bit). Efektywność kodu wynosi 99,21%, a warunek $H \le L < H + 1$ z twierdzenia Shannona o kodowaniu jest spełniony.

- Różnica $L - H$ bierze się stąd, że kod musi mieć całkowitą liczbę bitów, a $-\log_2 p$ zwykle nie jest liczbą całkowitą. Na wykresie słupki Huffmana są raz trochę wyższe, raz trochę niższe od ideału.
- Względem kodu o stałej długości (5 bitów) Huffman skraca zapis tekstu o 13,3%.
- Redundancja języka na poziomie pojedynczych liter wynosi ok. 12,3%: tyle „zbędnej” informacji wynika z nierównej częstości liter.
- Kod jest poprawny: zakodowany i zdekodowany korpus jest identyczny z oryginałem.

3. Laboratorium 3 – pokrycie tekstu, prawo Zipfa, graf sąsiedztwa

Zadanie: znaleźć wyrazy, które pokrywają 90% tekstu; sprawdzić prawo Zipfa; zbudować graf wyrazów sąsiadujących, zliczyć krawędzie wychodzące z każdego węzła i wyznaczyć rdzeń, czyli 100 wyrazów o największej liczbie sąsiadów.

3.1. Wyrazy pokrywające 90% tekstu

W prostych słowach

Ustawiamy wyrazy od najczęstszego i dodajemy ich udziały w tekście: „и” to 4,8% tekstu, „и” + „је” to już 8,8%, i tak dalej. Szukamy momentu, w którym suma przekroczy 90%. Liczba wyrazów do tego miejsca mówi, ile słów trzeba znać, żeby rozumieć 90% słów w tekście.

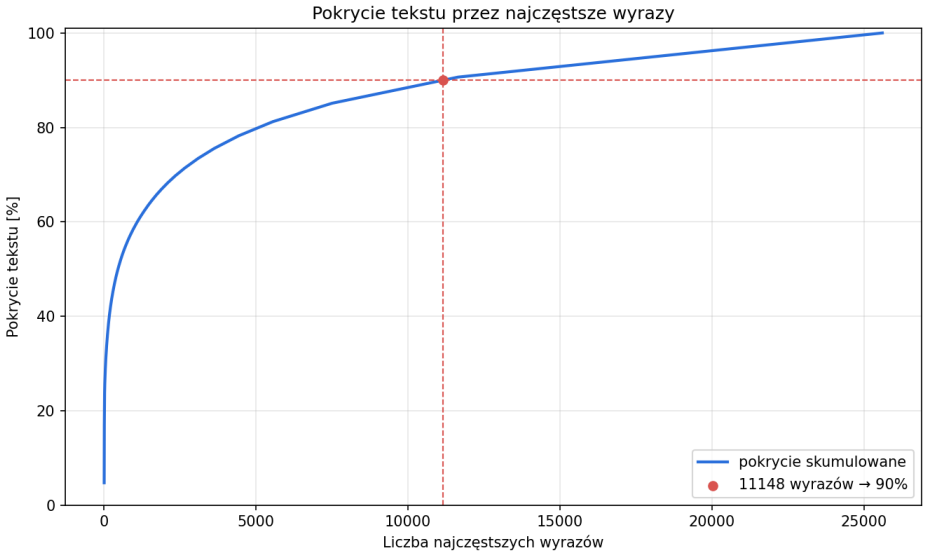
Korpus zawiera 149 373 wyrazów (tokenów) i 25 608 różnych wyrazów (typów). **Do pokrycia 90% tekstu potrzeba 11 148 najczęstszych wyrazów**, czyli 43,53% słownika. Aż 13 984 wyrazów (54,6%) występuje tylko raz.

Pokrycie tekstu	Liczba najczęstszych wyrazów	Udział w słowniku
25%	19	0,07%
50%	455	1,78%
75%	3 482	13,60%
80%	5 091	19,88%
90%	11 148	43,53%
95%	18 140	70,84%
99%	24 115	94,17%

Tabela 8. Liczba najczęstszych wyrazów potrzebna do pokrycia danej części tekstu.

Ranga	Wyraz	Liczność	Udział [%]	Skum. [%]	Ranga	Wyraz	Liczność	Udział [%]	Skum. [%]
1	и (i)	7099	4,7525	4,7525	11	а (a)	1110	0,7431	21,5621
2	је (jest)	6083	4,0724	8,8249	12	за (za / dla)	1059	0,7090	22,2711
3	у (w)	5270	3,5281	12,3530	13	који (który)	748	0,5008	22,7719
4	на (na)	2440	1,6335	13,9865	14	као (jako)	726	0,4860	23,2579
5	су (są)	2273	1,5217	15,5082	15	из (z)	715	0,4787	23,7366
6	се (się)	2214	1,4822	16,9904	16	до (do)	652	0,4365	24,1730
7	да (że / żeby)	1537	1,0290	18,0193	17	србије (Serbii)	586	0,3923	24,5653
8	године (roku)	1473	0,9861	19,0054	18	био (był)	427	0,2859	24,8512
9	од (od)	1438	0,9627	19,9681	19	по (po)	343	0,2296	25,0808
10	са (z)	1271	0,8509	20,8190	20	о (o)	342	0,2290	25,3098

Tabela 9. 20 najczęstszych wyrazów (z polskim znaczeniem). Pełna lista 11 148 wyrazów: wyniki/lab03/wyrazy_pokrycie_90.csv.



Rys. 4. Pokrycie tekstu przez najczęstsze wyrazy.

Wniosek: krzywa rośnie bardzo szybko na początku: zaledwie 19 wyrazów (głównie spójniki i przyimki) pokrywa 25% tekstu, a 455 wyrazów – połowę. Potem rośnie powoli, bo pozostała część tekstu składa się z rzadkich wyrazów, m.in. różnych form odmiany tego samego słowa i nazw własnych.

3.2. Prawo Zipfa

W prostych słowach

George Zipf zauważył, że w każdym dużym tekście drugi najczęstszy wyraz występuje mniej więcej 2 razy rzadziej niż pierwszy, trzeci 3 razy rzadziej itd. Wzorem: f(r) ≈ C / r, gdzie r to miejsce w rankingu, a f to częstość. Na wykresie w skali logarytmicznej (log-log) taka zależność jest linią prostą o nachyleniu −1.

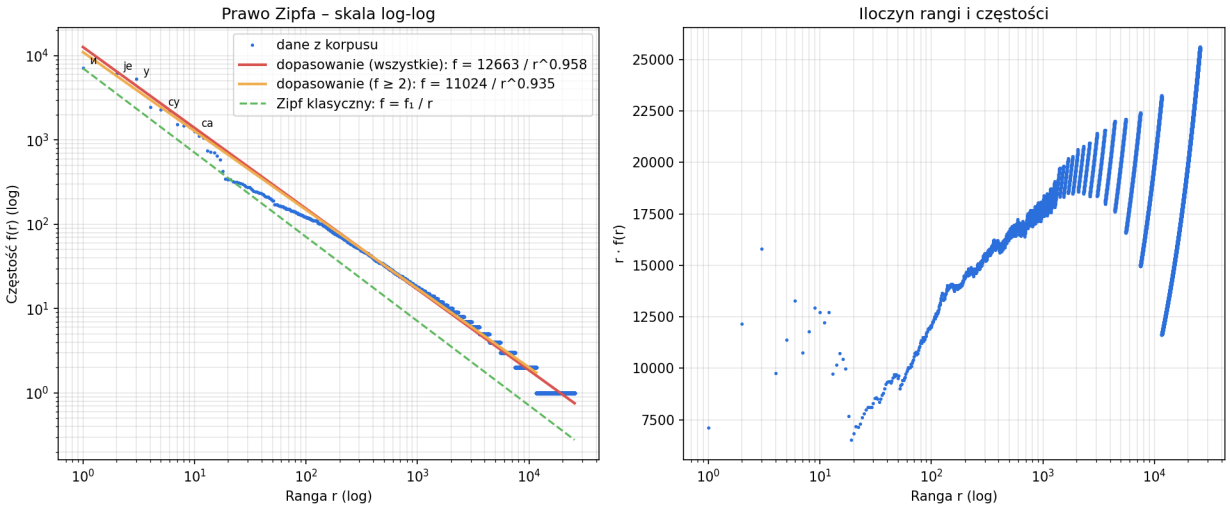
Ogólniej: $f(r) = C / r^s$. Po zlogarytmowaniu $\log f = \log C - s \cdot \log r$, więc s i C wyznaczyliśmy regresją liniową (metodą najmniejszych kwadratów, `numpy.polyfit`) na punktach (log r, log f).

Dopasowanie	Wykładnik s	C	R^2
wszystkie wyrazy (25 608)	**0,9584**	12 663,4	0,9652
bez hapaksów, f ≥ 2 (11 624)	0,9351		

Tabela 10. Parametry prawa Zipfa dla korpusu (R^2 – jakość dopasowania, 1 = idealne).

Ranga r	Wyraz	f(r)	r · f(r)	f z prawa Zipfa (f_1 / r)
1	и (i)	7099	7 099	7 099,0
2	је (jest)	6083	12 166	3 549,5
3	у (w)	5270	15 810	2 366,3
5	су (są)	2273	11 365	1 419,8
10	са (z)	1271	12 710	709,9
20	о (o)	342	6 840	354,9
50	државе (państwa)	190	9 500	142,0
100	један (jeden)	120	12 000	71,0
200	друге (drugie / inne)	72	14 400	35,5
500	освојио (zdobył)	33	16 500	14,2
1000	војно (wojskowo)	18	18 000	7,1

Tabela 11. Sprawdzenie prawa Zipfa dla wybranych rang: iloczyn r · f(r) powinien być w przybliżeniu stały.



Rys. 5. Prawo Zipfa: częstość w funkcji rangi w skali log-log (po lewej) i iloczyn r · f(r) (po prawej).

Wniosek: punkty układają się niemal na prostej ($R^2 = 0{,}965$), a wykładnik $s = 0{,}96$ jest bardzo bliski 1, więc **korpus spełnia prawo Zipfa**. Odchylenia widać dla kilku pierwszych rang (bardzo częste słowa funkcyjne и, у, је) i w ogonie, gdzie tysiące wyrazów mają tę samą małą częstość (poziome „schodki"). Lekkie spłaszczenie ($s < 1$) wynika z odmiany wyrazów w serbskim: србија, србије, србији, србију liczone są osobno. Ciekawostka: na 10 razy mniejszym korpusie (100 tys. znaków) wyszło $s = 0{,}68$. Prawo Zipfa dobrze widać dopiero na dużym tekście.

3.3. Graf wyrazów sąsiadujących i jego rdzeń

W prostych słowach

Każdy wyraz to punkt (węzeł). Jeśli dwa wyrazy choć raz stoją w tekście obok siebie, łączymy je kreską (krawędzią). Dla fragmentu „кнез лазар је подигао цркву" powstają krawędzie кнез–лазар, лазар–је, је–подигао, подигао–цркву. Wyraz, z którego wychodzi najwięcej krawędzi, sąsiaduje z największą liczbą różnych wyrazów, więc jest „centralny" dla języka. 100 takich wyrazów to rdzeń grafu.

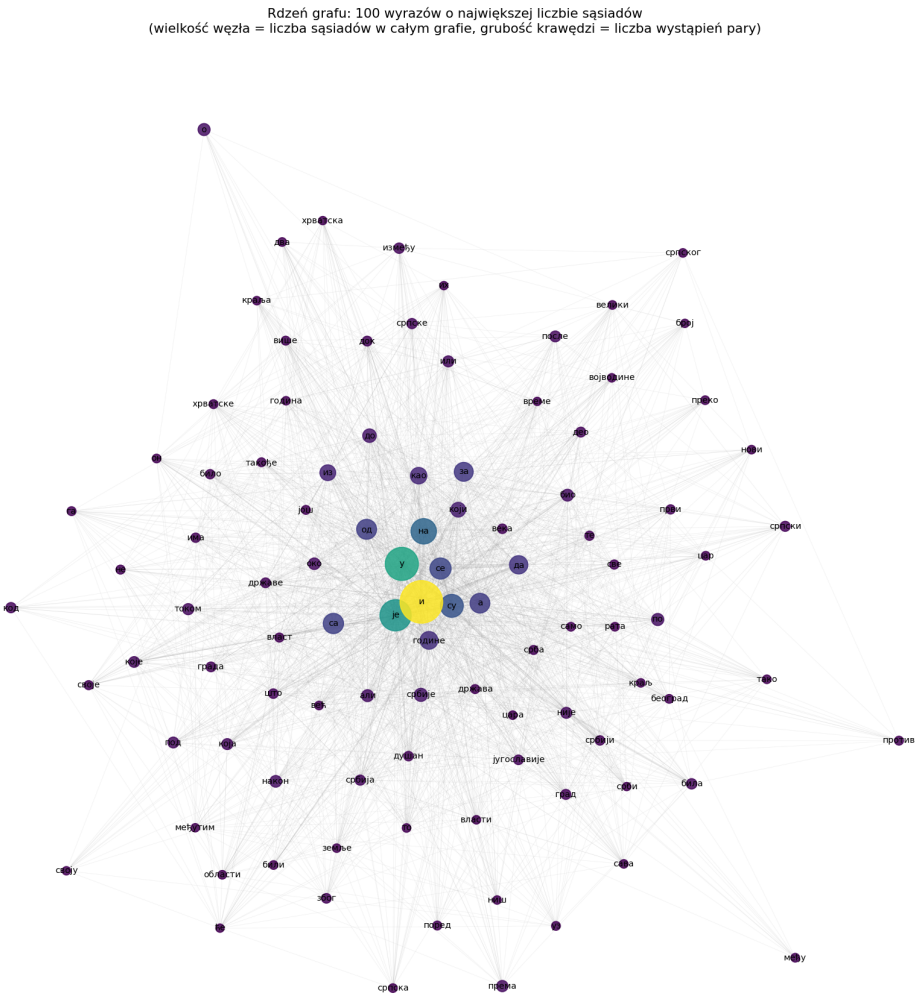
Krawędzie zliczyliśmy na dwa sposoby. **Krawędzie wychodzące** z wyrazu w to różne wyrazy występujące bezpośrednio po nim; **wchodzące** to wyrazy występujące bezpośrednio przed nim. Liczba wszystkich różnych sąsiadów to liczba krawędzi wychodzących z węzła w grafie nieskierowanym; według niej wyznaczyliśmy rdzeń. Graf zbudowano w `lab03_wyrazy_zipf_graf.py` (słowniki zbiorów sąsiadów, zapis do formatu GEXF biblioteką networkx).

Wielkość	Wartość
Liczba węzłów (różnych wyrazów)	25 608
Liczba krawędzi skierowanych (a → b)	99 265
Liczba krawędzi nieskierowanych (a — b)	95 855
Średnia / mediana liczby sąsiadów	7,49 / 2
Maksymalna liczba sąsiadów	6 800 (и (i))

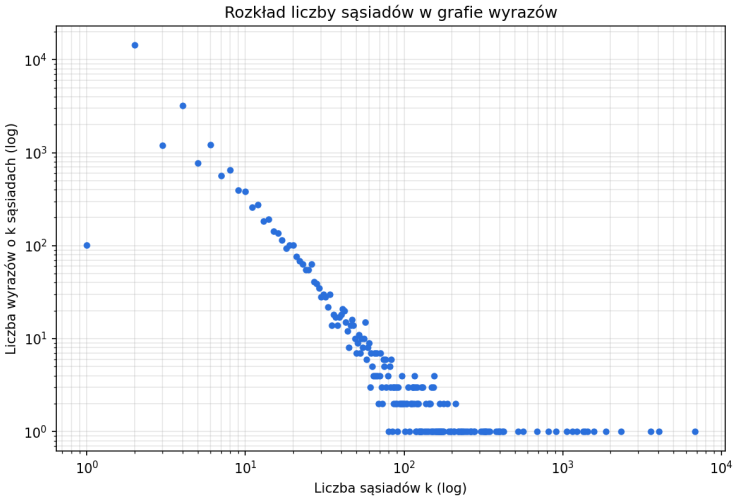
Tabela 12. Podstawowe parametry grafu sąsiedztwa.

Lp	Wyraz	Sąs.	Wych.	Wch.	Częst.	Lp	Wyraz	Sąs.	Wych.	Wch.	Częst.
1	и (i)	6800	3986	3773	7099	51	први (pierwszy)	191	91	106	173
2	у (w)	4059	1695	2617	5270	52	све (wszystko)	188	82	114	152
3	је (jest)	3602	2435	1835	6083	53	те (te / oraz)	188	82	113	131
4	на (na)	2328	1029	1401	2440	54	срба (Serbów)	178	79	106	204
5	су (są)	1878	1183	938	2273	55	своје (swoje)	178	94	88	119
6	се (się)	1572	1080	681	2214	56	власт (władza)	177	110	78	173
7	са (z)	1433	703	765	1271	57	не (nie)	176	85	93	140
8	од (od)	1382	714	749	1438	58	више (więcej)	172	94	85	159
9	а (a)	1351	587	830	1110	59	држава (państwo)	171	86	97	153
10	за (za / dla)	1228	607	663	1059	60	поред (obok / oprócz)	170	78	94	114
11	да (że / żeby)	1149	487	704	1537	61	још (jeszcze)	169	79	98	151
12	године (roku)	1064	719	441	1473	62	га (go)	168	95	80	150
13	као (jako)	904	389	535	726	63	против (przeciw)	168	77	97	146
14	из (z)	814	346	484	715	64	такође (także)	166	94	77	141
15	који (który)	691	131	567	748	65	хрватске (Chorwacji)	165	84	87	123
16	до (do)	562	334	271	652	66	уз (przy / wraz z)	163	75	89	116
17	србије (Serbii)	521	279	260	586	67	београд (Belgrad)	161	70	104	157
18	по (po)	423	207	225	343	68	само (tylko)	159	86	80	110
19	о (o)	417	233	188	342	69	међу (pośród)	156	63	93	118
20	али (ale)	402	104	305	341	70	он (on)	154	61	98	165
21	био (był)	397	221	189	427	71	били (byli)	154	81	78	145
22	након (po)	390	163	234	317	72	српска (serbska)	154	69	93	127
23	око (około)	379	202	197	317	73	преко (przez)	154	83	79	101
24	према (według / ku)	347	164	187	238	74	рата (wojny)	153	112	45	171
25	током (podczas)	336	139	207	268	75	цар (car)	153	79	83	167
26	или (lub)	328	169	186	211	76	велики (wielki)	153	51	104	151
27	није (nie jest)	326	156	172	276	77	области (obwodu / dziedziny)	152	86	74	114
28	која (która)	324	66	259	325	78	два (dwa)	151	87	69	128
29	под (pod)	318	128	193	280	79	тако (tak)	149	34	119	142
30	после (po)	314	152	165	243	80	српског (serbskiego)	149	50	103	137
31	које (które)	305	67	240	307	81	већ (już)	149	68	88	133
32	била (była)	277	173	114	290	82	нови (nowy)	148	41	110	171
33	између (między)	276	119	160	213	83	то (to)	146	91	70	216
34	код (u / przy)	266	135	138	197	84	сава (Sawa)	146	73	80	164
35	српске (serbskiej)	262	100	172	242	85	цара (cara)	145	77	71	163
36	србија (Serbia)	260	113	164	277	86	земље (kraju / ziemi)	145	79	75	107
37	српски (serbski)	252	118	142	237	87	власти (władzy)	144	87	59	128
38	град (miasto)	245	126	139	237	88	војводине (Wojwodiny)	144	82	65	126
39	део (część)	237	142	102	223	89	краљ (król)	143	63	89	168
40	док (podczas gdy)	233	40	197	228	90	година (rok / lat)	143	91	63	137
41	града (miasta)	230	116	123	233	91	краља (króla)	142	65	80	143
42	државе (państwa)	228	126	108	190	92	ниш (Nisz)	139	75	71	130
43	због (z powodu)	221	97	126	151	93	својој (swoją)	137	66	73	104
44	југославије (Jugosławii)	219	128	99	212	94	међутим (jednak)	137	64	76	81
45	што (co)	212	49	168	312	95	број (liczba)	135	82	56	122
46	века (wieku)	212	163	58	246	96	срби (Serbowie)	131	65	75	130
47	душан (Duszan)	208	81	136	258	97	ће (będzie)	131	83	51	129
48	србији (w Serbii)	206	159	50	300	98	их (ich)	131	66	70	120
49	има (ma)	198	106	97	162	99	хрватска (Chorwacja)	130	57	82	112
50	било (było)	197	122	84	202	100	време (czas)	129	112	20	157

Tabela 13. Rdzeń grafu – 100 wyrazów o największej liczbie sąsiadów (Sąs. – różni sąsiedzi, Wych./Wch. – krawędzie wychodzące/wchodzące, Częst. – liczba wystąpień).



Rys. 6. Podgraf rdzenia: wielkość węzła odpowiada liczbie sąsiadów w całym grafie, grubość krawędzi – liczbie wystąpień pary.



Rys. 7. Rozkład liczby sąsiadów (skala log-log): bardzo wiele wyrazów ma 1–2 sąsiadów, nieliczne – tysiące.

Wnioski.

- Rdzeń tworzą głównie **słowa funkcyjne**: spójniki (и, а, али), przyimki (у, на, од, са, за), formy czasownika бити (је, су) i zaimki. Łączą się one z niemal każdym wyrazem, dlatego mają tysiące sąsiadów.

- Wyrazy rdzenia to 35,1% wszystkich wyrazów tekstu, a 88 ze 100 z nich należy też do 100 najczęstszych wyrazów: im częstszy wyraz, tym więcej ma sąsiadów.
- Ranking według samych krawędzi wychodzących (skierowanych) daje rdzeń wspólny z powyższym w 83 wyrazach na 100, więc wybór sposobu liczenia krawędzi niewiele zmienia.
- Rozkład liczby sąsiadów jest potęgowy (prosta w skali log-log): mediana to 2 sąsiadów, a maksimum to kilka tysięcy. Taki graf nazywa się **bezskalowym**: kilka węzłów-hubów łączy całą sieć, podobnie jak w sieciach społecznych czy w internecie.

4. Laboratorium 4 – sieć Kohonena: rzeczowniki w przestrzeni czasowników przechodnich

Zadanie: wybrać 50 rzeczowników i 100 czasowników przechodnich. Dla każdego rzeczownika wygenerować wektor o 100 składowych (kolokacje z czasownikami) – to zbiór treningowy. Nauczyć na nim sieć Kohonena i przedstawić rzeczowniki w 100-wymiarowej przestrzeni czasowników. Zbadać, jak wynik zależy od wyboru czasowników, np. biorąc 100 najczęstszych czasowników korpusu.

W prostych słowach

Rzeczownik można poznać po tym, z jakimi czasownikami się łączy. „Decyzję” i „ustawę” się *podejmuje/uchwala* (донети), „wojnę” i „walkę” się *prowadzi* (водити), a „miasto” i „medal” się *zdobywa* (освојити). Zapisujemy więc dla każdego rzeczownika 100 liczb: ile razy był dopełnieniem każdego ze 100 czasowników. Te 100 liczb to punkt w przestrzeni 100-wymiarowej, której osiami są czasowniki. Człowiek nie zobaczy 100 wymiarów, więc sieć Kohonena „spłaszcza” tę przestrzeń do mapy 10 × 10, tak żeby rzeczowniki o podobnych wektorach leżały obok siebie.

4.1. Analiza składniowa korpusu

Żeby wiedzieć, które czasowniki są przechodnie i który rzeczownik jest dopełnieniem którego czasownika, potrzebna jest analiza gramatyczna zdań. Użyto biblioteki **stanza** (Stanford NLP) z modelem języka serbskiego (treebank UD Serbian-SET, standard Universal Dependencies). Skrypt `lab04_parsowanie.py`:

1. wczytuje tekst surowy z lab 1 (z interpunkcją, bo potrzebne są granice zdań),
2. zamienia cyrylicę na łacinkę, bo model był uczony na serbskiej łacince (odpowiedniość liter jest 1:1, np. љ → lj, њ → nj, џ → dž),
3. dla każdego wyrazu ustala część mowy, lemat (formę podstawową) i wyraz nadrzędny wraz z relacją, np. *obj* = dopełnienie bliższe,
4. przepisuje wynik z powrotem na cyrylicę i zapisuje go do dane/korpus_sparsowany.tsv.gz.

Wynik: 7 949 zdań, 178 663 tokenów (razem z interpunkcją), 1 679 różnych czasowników i 5 203 różnych rzeczowników, **3 890 par czasownik → dopełnienie rzeczownikowe**. Przykład: w zdaniu „Кнез Лазар је подигао цркву и водио војску” parser znajduje pary obj(подићи, црква) i obj(водити, војска). Dzięki lematyzacji formy цркву, цркве, црквом liczą się jako jeden rzeczownik црква.

4.2. Wybór 100 czasowników przechodnich i 50 rzeczowników

1. Dla każdego czasownika liczymy, ile razy ma w korpusie dopełnienie rzeczownikowe. Liczymy też dopełnienia współrzędne: w „подигао цркву и кућу” oba rzeczowniki są dopełnieniami.
2. Czasownik uznajemy za **przechodni**, jeśli dopełnienie ma w co najmniej 20% wystąpień. Odrzuca to błędy parsera, np. постати (stać się; „постао је краљ” – „został królem”), постојати (istnieć) i вратити (najczęściej zwrotne вратити се – wrócić).
3. Pomijamy czasowniki posiłkowe i modalne (бити, хтети, моћи, морати, требати).
4. Bierzemy **100 czasowników** o największej liczbie dopełnień, a potem **50 rzeczowników**, które najczęściej są dopełnieniem tych czasowników. Dzięki temu każdy wektor ma dane.

Lp	Czasownik	Dop.	Wyst.	Lp	Czasownik	Dop.	Wyst.
1	имати (mieć)	364	393	51	напустити (opuścić)	14	23
2	чинити (stanowić / czynić)	100	142	52	примити (przyjąć / otrzymać)	13	21
3	представљати (przedstawiać / stanowić)	99	99	53	променити (zmienić)	13	11
4	добити (otrzymać)	79	98	54	обновити (odnowić)	13	21
5	освојити (zdobyć)	58	95	55	потписати (podpisać)	13	19
6	обухватати (obejmować)	52	46	56	осетити (odczuć)	13	11
7	донети (przynieść / uchwalić)	47	51	57	пружати (stawiać / udzielać)	12	15
8	послати (wysłać)	41	57	58	признавати (uznawać)	12	15
9	дати (dać)	37	51	59	предати (przekazać / poddać)	12	28
10	прихватити (przyjąć)	31	41	60	штитити (chronić)	12	9
11	заузимати (zajmować)	30	43	61	издати (wydać)	12	16
12	признати (uznać)	28	42	62	називати (nazywać)	11	33
13	водити (prowadzić)	27	49	63	сачињавати (tworzyć / stanowić)	11	12
14	преузети (przejąć)	27	30	64	пружити (stawić / udzielić)	11	13
15	прогласити (ogłosić / proklamować)	27	44	65	уживати (cieszyć się (czymś))	11	10
16	наставити (kontynuować)	26	47	66	мењати (zmieniać)	11	14
17	оставити (zostawić)	26	42	67	усвојити (przyjąć / uchwalić)	11	12
18	повратити (odzyskać)	24	23	68	поставити (postawić / ustanowić)	10	17
19	заузети (zająć)	24	57	69	наћи (znaleźć)	10	44
20	стећи (zdobyć / nabyć)	23	27	70	настављати (kontynuować)	10	19
21	формирати (utworzyć)	23	38	71	постићи (osiągnąć)	10	18
22	извршити (wykonać / dokonać)	23	25	72	узети (wziąć)	10	15
23	носити (nosić)	22	25	73	одржавати (utrzymywać)	10	38
24	користити (używać)	22	43	74	изазивати (wywoływać)	10	11
25	доносити (przynosić / uchwalać)	22	17	75	преузимати (przejmować)	10	13
26	основати (założyć)	21	34	76	бирати (wybierać)	10	26
27	изгубити (stracić)	21	25	77	потврдити (potwierdzić)	10	9
28	подићи (wznieść / podnieść)	21	28	78	склопити (zawrzeć)	10	12
29	вршити (pełnić / sprawować)	21	24	79	покривати (pokrywać)	9	9
30	давати (dawać)	20	21	80	насељавати (zamieszkiwać)	9	18
31	организовати (organizować)	19	30	81	износити (wynosić)	9	47
32	добијати (otrzymywać)	19	27	82	карактерисати (charakteryzować)	9	9
33	покретати (wszczynać / uruchamiać)	18	19	83	увести (wprowadzić)	9	15
34	однети (odnieść)	17	19	84	освајати (zdobywać)	9	27
35	укључивати (obejmować / włączać)	17	20	85	започети (rozpocząć)	9	16
36	нудити (oferować)	17	10	86	ојачати (wzmocnić)	9	15
37	доживети (przeżyć / doznać)	16	19	87	чувати (strzec / przechowywać)	9	13
38	обављати (wykonywać)	16	23	88	поседовати (posiadać)	9	12
39	повезивати (łączyć)	16	33	89	означити (oznaczyć)	9	10
40	задржати (zachować)	16	29	90	држати (trzymać)	9	21
41	изазвати (wywołać)	16	16	91	подразумевати (oznaczać / zakładać)	9	10
42	створити (stworzyć)	15	20	92	пратити (śledzić / towarzyszyć)	9	10
43	тражити (szukać / żądać)	15	31	93	убити (zabić)	9	17
44	покренути (wszcząć / uruchomić)	15	19	94	завршити (zakończyć)	9	18
45	напасти (zaatakować)	15	40	95	именовати (mianować)	9	11
46	успоставити (ustanowić)	14	16	96	одржати (utrzymać / przeprowadzić)	9	23
47	садржати (zawierać)	14	12	97	испунити (spełnić)	9	8
48	остварити (osiągnąć)	14	24	98	проглашавати (ogłaszać)	8	13
49	искористити (wykorzystać)	14	21	99	проширити (rozszerzyć)	8	25
50	немати (nie mieć)	14	26	100	претрпети (ponieść / doznać)	8	9

Tabela 14. 100 wybranych czasowników przechodnich (Dop. – liczba dopełnień rzeczownikowych, Wyst. – liczba wystąpień czasownika).

Lp	Rzeczownik	Jako dop.	Wyst.	Lp	Rzeczownik	Jako dop.	Wyst.
1	власт (władza)	43	340	26	подршка (poparcie)	12	34
2	део (część)	35	453	27	повеља (przywilej / karta)	12	54
3	име (imię / nazwa)	34	172	28	помоћ (pomoc)	11	76
4	град (miasto)	34	686	29	функција (funkcja)	11	22
5	држава (państwo)	27	459	30	манастир (monaster)	11	78
6	титула (tytuł)	26	84	31	рат (wojna)	10	385
7	одлука (decyzja)	26	111	32	живот (życie)	10	105
8	право (prawo)	23	96	33	мир (pokój)	10	60
9	медаља (medal)	23	36	34	црква (cerkiew)	10	190
10	независност (niepodległość)	21	84	35	борба (walka)	9	101
11	становник (mieszkaniec)	21	136	36	закон (ustawa / prawo)	9	89
12	територија (terytorium)	18	236	37	успех (sukces)	9	22
13	статус (status)	18	44	38	резултат (wynik)	9	45
14	назив (nazwa)	17	98	39	син (syn)	9	150
15	улога (rola)	17	31	40	положај (położenie / pozycja)	9	63
16	већина (większość)	16	78	41	република (republika)	9	310
17	место (miejsce)	16	126	42	површина (powierzchnia)	9	57
18	област (obwód / dziedzina)	15	206	43	скупштина (zgromadzenie / parlament)	9	137
19	победа (zwycięstwo)	15	52	44	политика (polityka)	9	62
20	утицај (wpływ)	15	68	45	култура (kultura)	8	117
21	земља (kraj / ziemia)	14	286	46	пораз (porażka)	8	38
22	аутономија (autonomia)	14	36	47	отпор (opór)	8	43
23	војска (wojsko)	13	240	48	граница (granica)	8	120
24	традиција (tradycja)	13	42	49	преговор (negocjacje)	8	49
25	подручје (obszar)	13	196	50	услов (warunek)	8	31

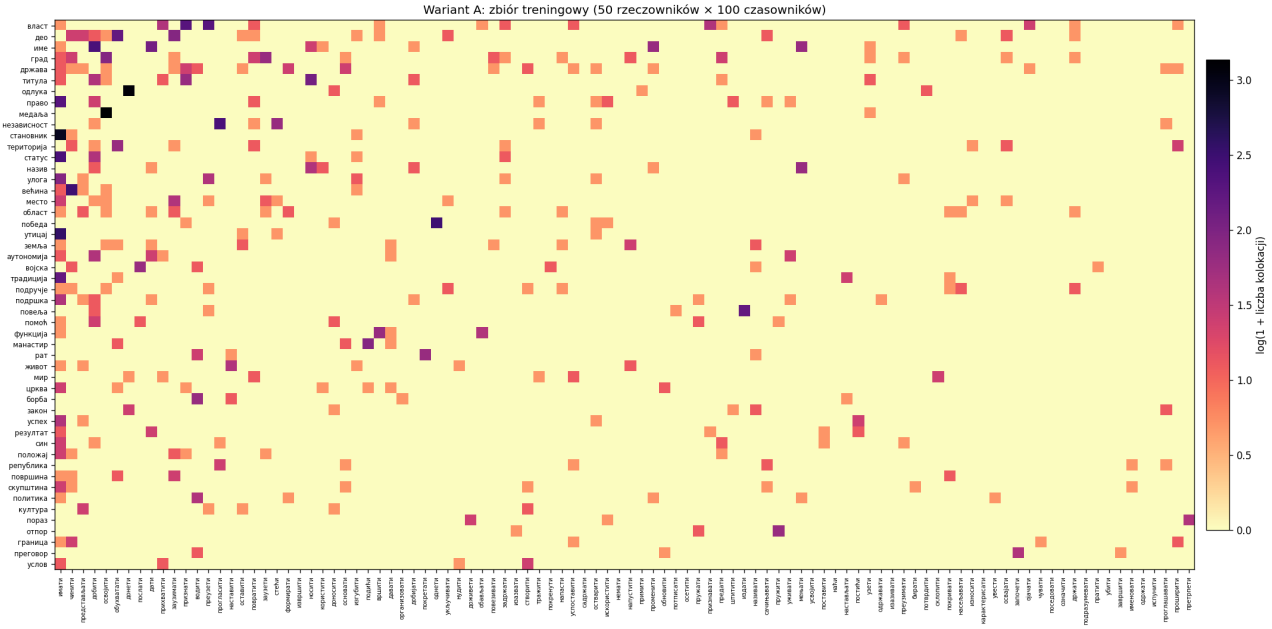
Tabela 15. 50 wybranych rzeczowników (Jako dop. – ile razy były dopełnieniem wybranych czasowników).

4.3. Zbiór treningowy – wektory o 100 składowych

Każdy rzeczownik zamieniamy na wektor $x = (x_1, \ldots, x_{100})$, gdzie x_j to liczba wystąpień rzeczownika jako dopełnienia j-tego czasownika. Przykład – wektor rzeczownika **одлука** (decyzja) ma 4 niezerowe składowe: донети (przynieść / uchwalić) = 21, доносити (przynosić / uchwalać) = 2, потврдити (potwierdzić) = 2, примити (przyjąć / otrzymać) = 1; pozostałe składowe to zera.

Cała macierz 50 × 100 zawiera 762 kolokacji; niezerowych jest 338 pól (6,8%), bo każdy rzeczownik łączy się tylko z kilkoma czasownikami. Przed uczeniem wektory przygotowano w dwóch krokach:

- **log(1 + x)** – zmniejsza wpływ bardzo częstych par (np. одлука + донети = 21), żeby jedna para nie zdominowała wektora,
- **normalizacja do długości 1** – porównujemy „profil” czasowników, a nie to, jak często rzeczownik w ogóle występuje (rzadki i częsty rzeczownik z tym samym profilem są podobne).

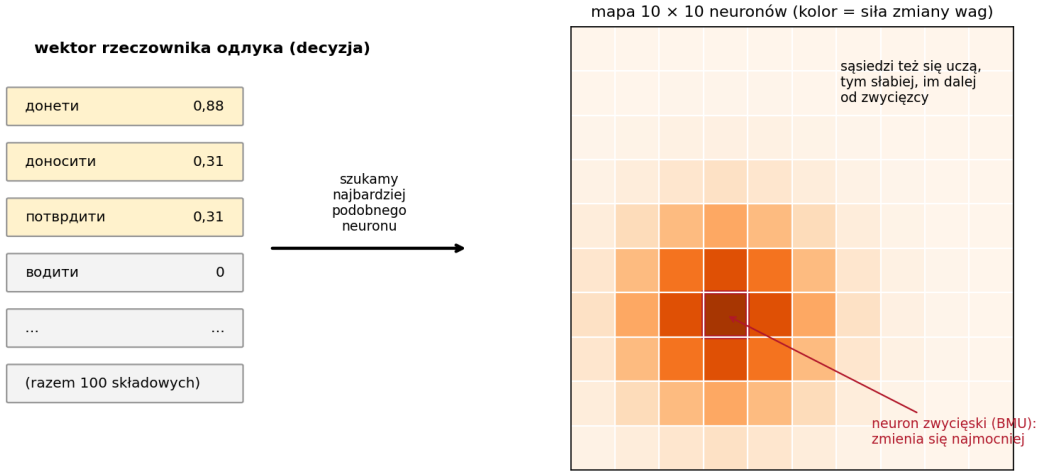


Rys. 8. Zbiór treningowy: 50 rzeczowników (wiersze) × 100 czasowników (kolumny); ciemniejszy kolor = więcej kolokacji.

4.4. Sieć Kohonena – jak działa i jak ją nauczyliśmy

W prostych słowach

Sieć Kohonena to siatka 10 × 10 „szufladek” (neuronów). Każda szufladka ma własny wektor 100 liczb, czyli wzorcowy profil czasowników. Uczenie polega na powtarzaniu jednego kroku: losujemy rzeczownik, szukamy szufladki najbardziej do niego podobnej (zwycięzcy) i przesuwamy jej wektor trochę w stronę tego rzeczownika. Sąsiednie szufladki też się przesuwają, ale słabiej. Po tysiącach powtórzeń sąsiednie szufladki mają podobne profile, więc podobne rzeczowniki trafiają w pobliskie miejsca mapy. Przypomina to układanie książek na półkach tak, żeby podobne tematy stały obok siebie.

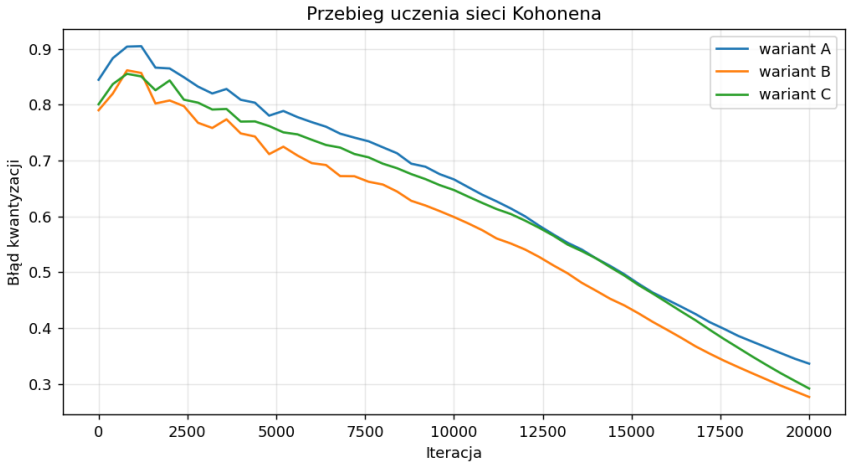


Rys. 9. Jeden krok uczenia sieci Kohonena: wektor rzeczownika (po lewej) przyciąga neuron zwycięski i, słabiej, jego sąsiadów (po prawej).

Sieć zaimplementowano samodzielnie w numpy (klasa `KohonenSOM` w `lab04_kohonen.py`). Parametry i algorytm:

- siatka 10 × 10 = 100 neuronów; każdy neuron ma wektor wag $w \in R^{100}$,
- inicjalizacja wag na płaszczyźnie dwóch głównych kierunków zmienności danych (PCA), dzięki czemu wynik jest powtarzalny,

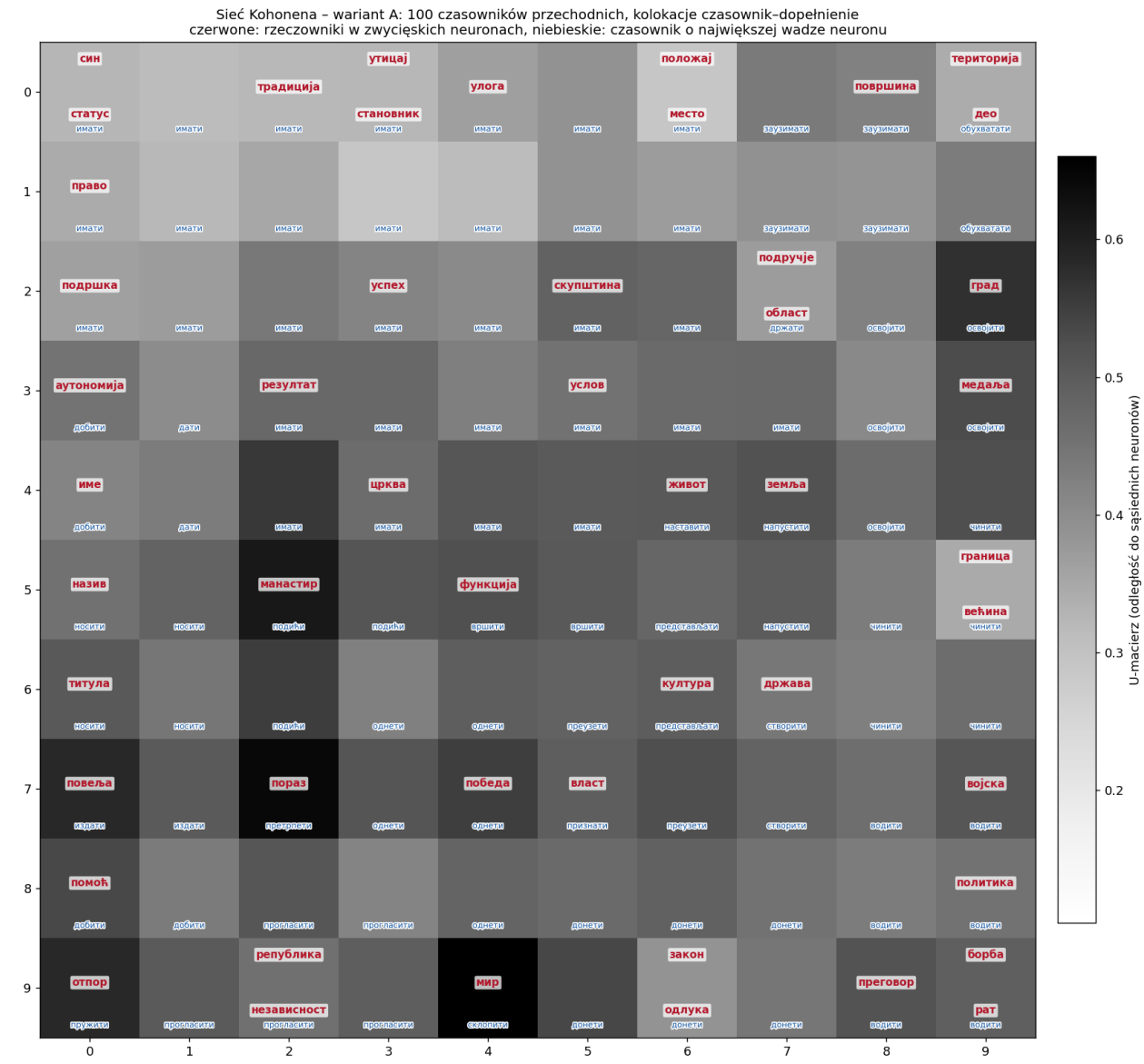
- 20 000 kroków; w każdym losujemy wektor x i znajdujemy neuron zwycięski (BMU) – o najmniejszej odległości euklidesowej $\|x - w\|$,
- aktualizacja wag: $w \leftarrow w + \eta(t) \cdot h(t) \cdot (x - w)$, gdzie $h = \exp(-d^2 / 2\sigma(t)^2)$ to „siła sąsiedztwa”, a d to odległość neuronu od zwycięzcy na siatce,
- współczynnik uczenia η maleje z 0,5 do 0,01, a promień sąsiedztwa σ z 5 do 0,5: najpierw mapa układa się z grubsza, potem dopracowuje szczegóły,
- ocena mapy: **błąd kwantyzacji** (średnia odległość wektora od jego BMU – im mniejszy, tym lepiej neurony odwzorowują dane) i **błąd topograficzny** (odsetek wektorów, których dwa najlepsze neurony nie sąsiadują – 0% oznacza zachowane sąsiedztwo).



Rys. 10. Spadek błędu kwantyzacji w trakcie uczenia (trzy warianty z p. 4.7).

4.5. Wynik – mapa rzeczowników

Na mapie każdy rzeczownik (czerwony) stoi w swoim neuronie zwycięskim. Pod nim, na niebiesko, podpisano czasownik, który ma w danym neuronie największą wagę. Tło to **U-macierz**: jasne pola oznaczają, że neuron jest podobny do sąsiadów (wnętrze skupienia), a ciemne, że się od nich różni (granica między skupieniami).



Rys. 11. Mapa Kohonena – wariant A (błąd kwantyzacji 0,336, błąd topograficzny 0,0%).

Rzeczowniki, które trafiły do tego samego neuronu, mają najbardziej podobne wektory: део (część), територија (terytorium); одлука (decyzja), закон (ustawa / prawo); независност (niepodległość), република (republika); становник (mieszkaniec), утицај (wpływ); статус (status), син (syn); већина (większość), граница (granica); место (miejsce), положај (położenie / pozycja); област (obwód / dziedzina), подручје (obszar); рат (wojna), борба (walka).

Interpretacja mapy. Sieć sama, bez żadnych etykiet, podzieliła rzeczowniki na obszary wyznaczone przez czasowniki, z którymi się łączą:

• **górne wiersze** – rejon czasownika *имати* (mieć): статус, традиција, становник, утицај, улога, право, подршка, успех („mieć status, tradycję, mieszkańców, wpływ, rolę, prawo, poparcie, sukces”);

• **prawy górny róg** – rzeczowniki przestrzenne z czasownikami *заузимати* (zajmować) i *обухватати* (obejmować): место, положај, површина, део, територија, подручје, област;

• **lewa kolumna** – rzeczy, które się otrzymuje, nadaje lub nosi (*добити, дати, носити, издати, пружити*): аутономија, име, назив, титула, повеља, помоћ, отпор;

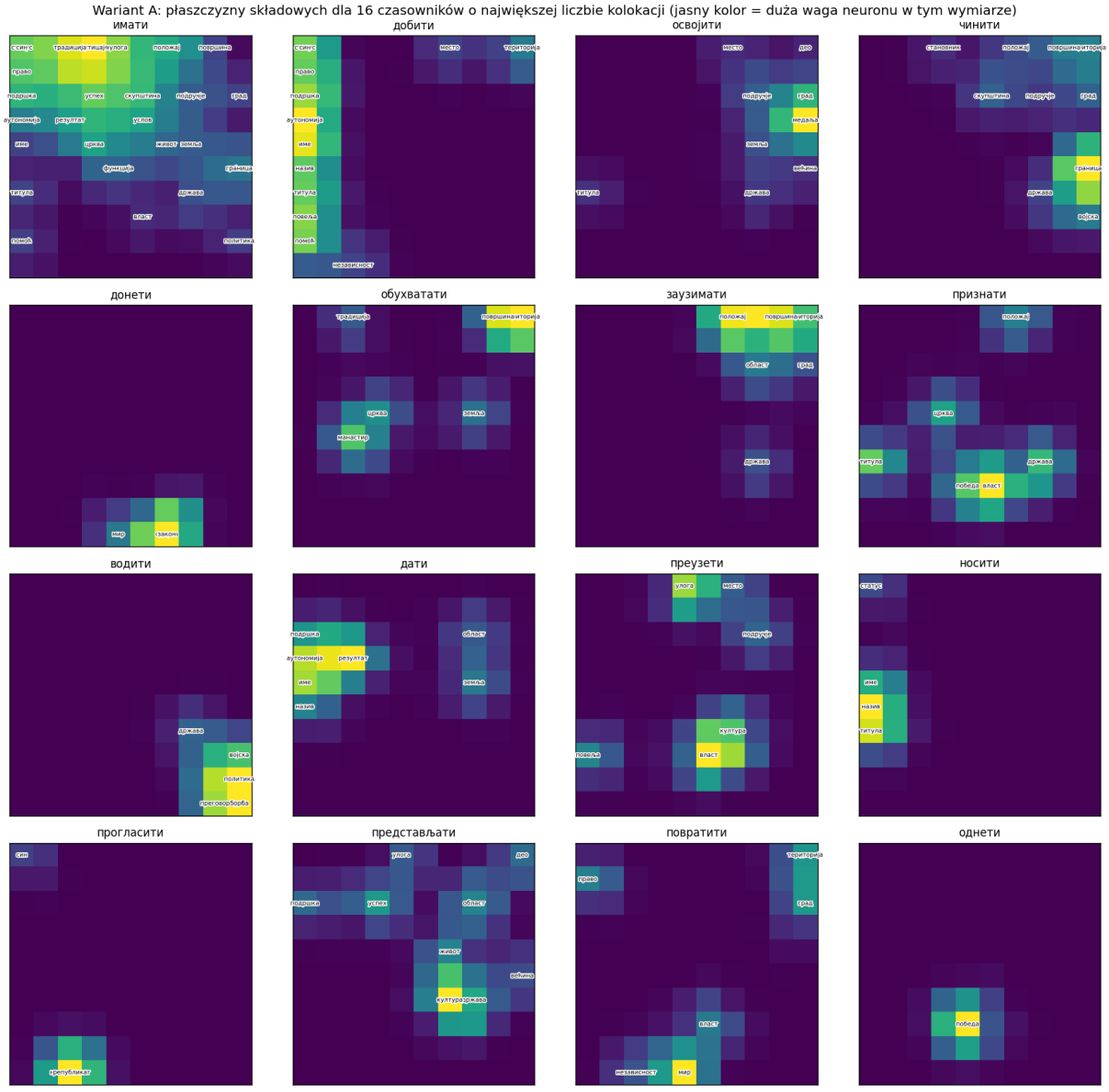
• **dolny wiersz** – działania polityczne i wojskowe: независност и република (*прогласити* – ogłosić), одлука и закон (*донети* – podjąć, uchwalić), мир (*склопити* – zawrzeć), преговор, борба, рат, политика, војска (*водити* – prowadzić);

• **środek dołu mapy** – пораз (*претрпети* – ponieść), победа (*однети* – odnieść), власт (*преузети* – przejąć).

Ciekawy przypadek to град (miasto) і медаља (medal) leżące obok siebie. Łączy je czasownik *освојити*, który znaczy zarówno „zdobyć miasto”, jak i „zdobyć medal”. Mapa grupuje więc rzeczowniki według zachowania składniowego (wspólnych czasowników), a nie według samego znaczenia. Błąd topograficzny wynosi 0%: rzeczowniki podobne w przestrzeni 100D leżą na mapie obok siebie.

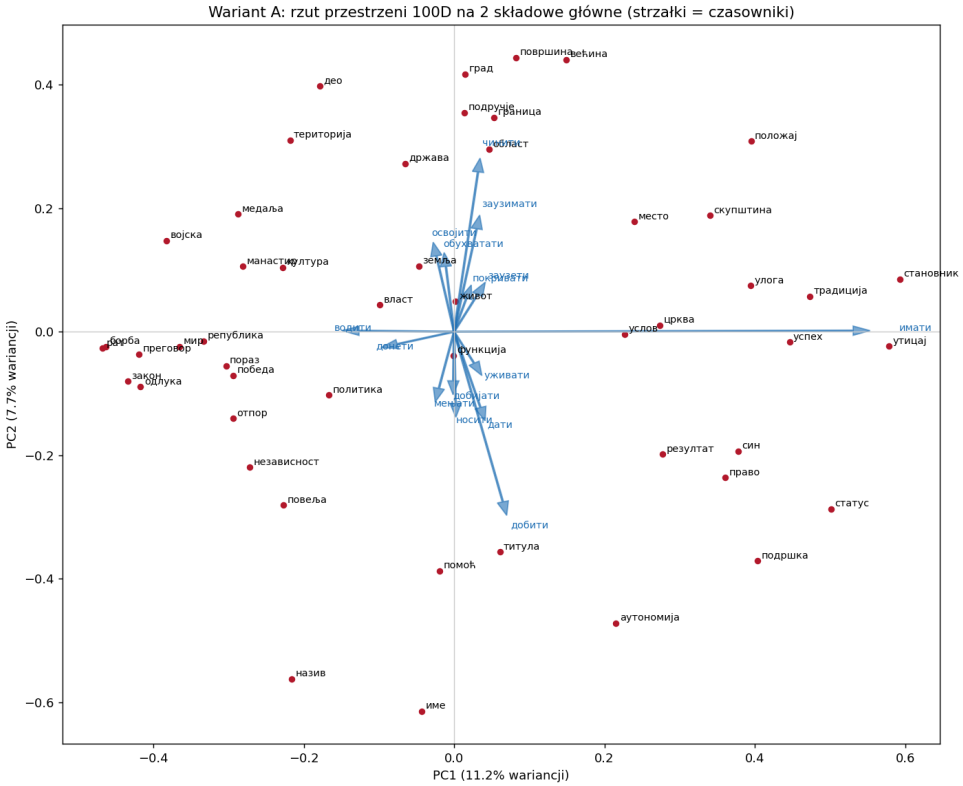
4.6. Gdzie są czasowniki w przestrzeni 100D

Każda oś 100-wymiarowej przestrzeni to jeden czasownik. Dla każdej osi można narysować **płaszczyznę składowej**, czyli wartość tej jednej wagi we wszystkich 100 neuronach. Jasny obszar to rejon mapy „należący” do danego czasownika: leżą tam rzeczowniki, które się z nim łączą. Rysunek pokazuje 16 czasowników o największej liczbie kolokacji.



Rys. 12. Płaszczyzny składowych – położenie wybranych czasowników (osi przestrzeni 100D) na mapie.

Widać, że czasowniki zajmują **różne, zwarte obszary** mapy: имати – górne wiersze, заузимати і обухватати – prawy górny róg, добити і носити – lewą kolumnę, водити і донети – dół mapy. Mapa jest więc dwuwymiarowym „rzutem” przestrzeni czasowników, w którym sąsiedztwo zostało zachowane.



Rys. 13. Dla porównania: zwykły rzut liniowy PCA tych samych wektorów 100D na płaszczyznę. Strzałki to kierunki osi najważniejszych czasowników.

Dla porównania pokazano rzut PCA. Dwie najważniejsze osie zachowują tylko 18,8% informacji (wariancji), a pierwsza oś to w praktyce jeden czasownik – имати. Rzut liniowy gubi więc większość struktury, a część punktów się nakłada. Sieć Kohonena jest nieliniowa: korzysta ze wszystkich 100 wymiarów i układa rzeczowniki po całej mapie, zachowując sąsiedztwo.

4.7. Czy wynik zależy od wyboru czasowników?

Ten sam zbiór 50 rzeczowników nauczono w trzech wariantach, przy identycznych parametrach sieci:

- **Wariant A** – 100 czasowników przechodnich, kolokacje czasownik–dopełnienie.
- **Wariant B** – 100 najczęstszych czasowników, kolokacje czasownik–dopełnienie.
- **Wariant C** – 100 czasowników przechodnich, kolokacje w oknie ±5 wyrazów.

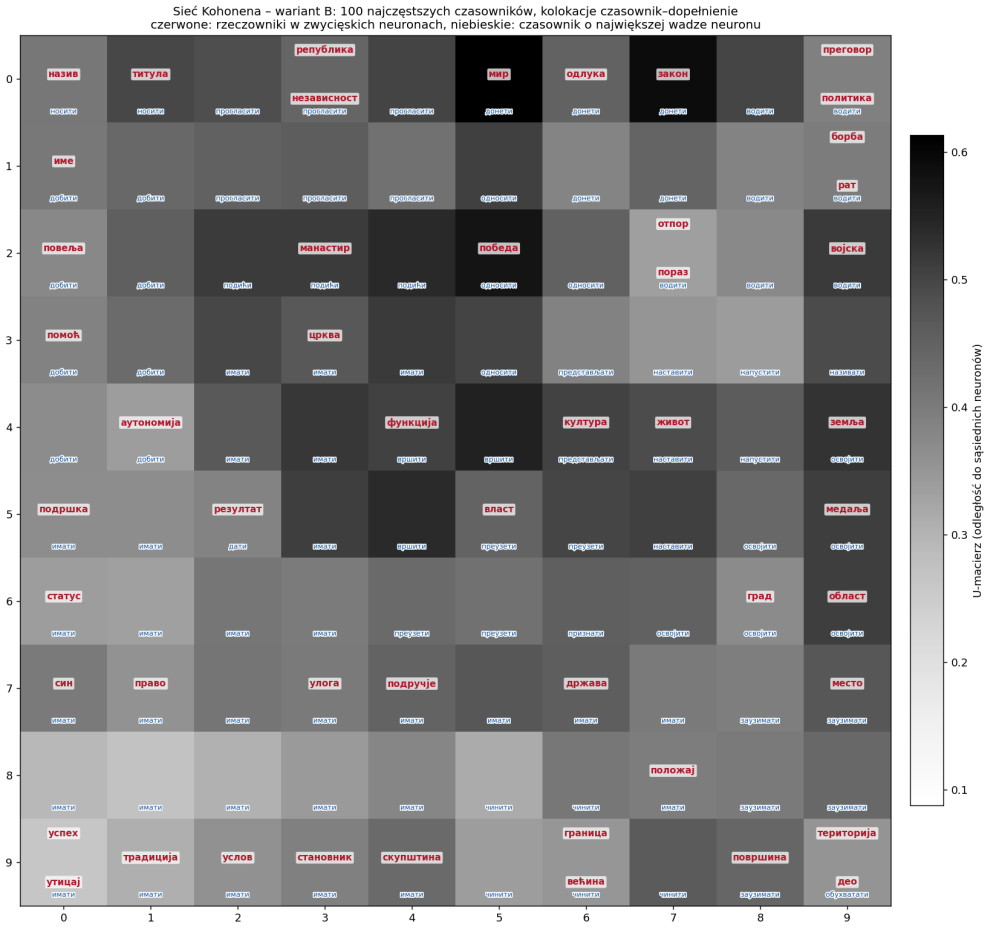
Warianty A i B mają 49 wspólnych czasowników na 100. W wariancie B są też częste czasowniki nieprzechodnie (налазити се, постати, доћи, живети, ићи, умрети), które prawie nie mają dopełnień rzeczownikowych.

Wariant	Suma kolokacji	Wypełnienie macierzy	Rzeczowniki z zerowym wektorem	Błąd kwantyzacji	Błąd topograficzny	Zajęte neurony
A	762	6,8%	0	0,336	0,0%	41
B	617	5,3%	пораз, отпор	0,276	0,0%	43
C	1 784	16,5%	0	0,292	0,0%	49

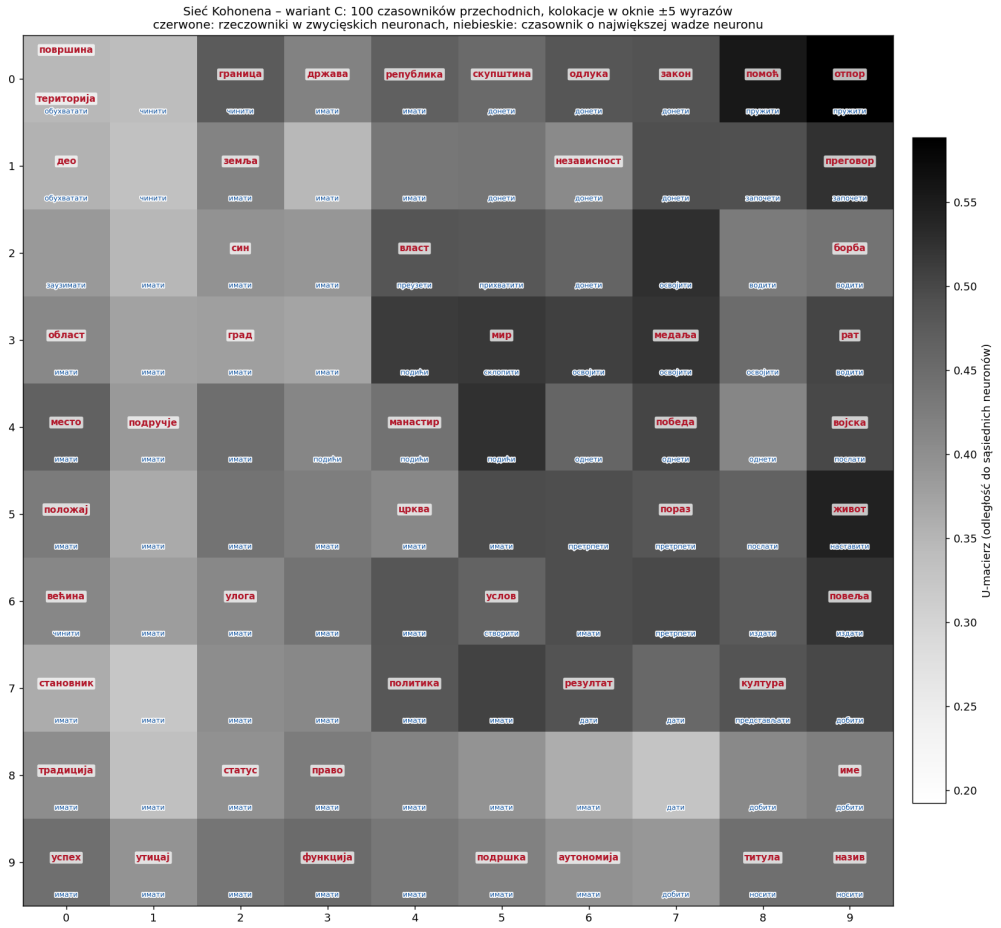
Tabela 16. Porównanie wariantów.

Porównanie map	Korelacja odległości (Spearman)	Wspólni sąsiedzi (5 najbliższych)
A – B	0,701	48,0%
A – C	0,405	30,0%
B – C	0,429	30,0%

Tabela 17. Podobieństwo map. Korelacja 1 oznacza identyczny układ wzajemnych odległości rzeczowników na mapie, 0 – brak związku.



Rys. 14. Mapa Kohonena – wariant B (100 najczęstszych czasowników).



Rys. 15. Mapa Kohonena – wariant C (kolokacje w oknie ±5 wyrazów).

Wnioski z porównania. Wynik wyraźnie zależy od wyboru czasowników:

• **A i B** mają wspólną połowę czasowników, w tym te o największej liczbie kolokacji (имати, донети, водити, прогласити, освојити). Dlatego ogólny układ mapy jest podobny (korelacja odległości ok. 0,7), ale pokrywa się tylko ok. połowa najbliższych sąsiadów. Czasowniki nieprzechodnie z wariantu B nie wnoszą informacji o dopełnieniu, a rzeczowniki пораз i отпор mają w B zerowe wektory, bo ich jedyne czasowniki (претрпети, пружити) nie należą do 100 najczęstszych. Trafiają więc do jednego neuronu z braku danych, a nie z podobieństwa.

• **C** (okno ±5 wyrazów zamiast relacji składniowej) daje gęstszą, ale bardziej zaszumioną macierz. Okno łapie też podmioty i wyrazy niezwiązane z czasownikiem: np. град dostaje 22 kolokacje z *имати* ze zdań typu „град има … становника” (miasto ma … mieszkańców), w których jest podmiotem, a nie dopełnieniem. Mapa C najmniej przypomina pozostałe, choć najsilniejsze pary (одлука–закон, рат–борба, територија–површина) nadal leżą obok siebie.

Najbardziej wiarygodna jest więc mapa A, w której wymiarami są czasowniki faktycznie przechodnie, a kolokacje pochodzą z relacji czasownik–dopełnienie.

5. Sprawdzenie poprawności wyników

Skrypt `sprawdz_wyniki.py` nie korzysta z kodu laboratoriów: każdą wartość liczy od nowa, najprostszą możliwą metodą, i porównuje ją z plikami wynikowymi. Poniżej wynik jego działania.

Lab	Co sprawdzamy	Wynik
1	w korpusie są tylko litery serbskiego alfabetu i spacje	**OK**
1	brak cyfr (także np. ², ½)	**OK**
1	brak interpunkcji i symboli	**OK**
1	brak liter łacińskich i greckich	**OK**
1	wszystkie litery są małe	**OK**
1	brak podwójnych spacji, nowych linii i spacji na brzegach	**OK**
1	występują wszystkie 30 liter alfabetu (30/30)	**OK**
1	tekst surowy zawierał znaki, które usunięto	**OK**
2	[same litery] liczności liter zgodne z korpusem	**OK**
2	[same litery] udziały procentowe sumują się do 100% (100.0001%)	**OK**
2	[same litery] kod Huffmana jest prefiksowy (żaden kod nie jest początkiem innego)	**OK**
2	[same litery] nierówność Krafta: suma 2^(-długość) = 1	**OK**
2	[same litery] średnia długość L jest minimalna (= niezależny Huffman) (L = 4.3368)	**OK**
2	[same litery] H ≤ L < H + 1 (H = 4.3024, L = 4.3368)	**OK**
2	[litery + spacja] liczności liter zgodne z korpusem	**OK**
2	[litery + spacja] udziały procentowe sumują się do 100% (99.9999%)	**OK**
2	[litery + spacja] kod Huffmana jest prefiksowy (żaden kod nie jest początkiem innego)	**OK**
2	[litery + spacja] nierówność Krafta: suma 2^(-długość) = 1	**OK**
2	[litery + spacja] średnia długość L jest minimalna (= niezależny Huffman) (L = 4.2889)	**OK**
2	[litery + spacja] H ≤ L < H + 1 (H = 4.2588, L = 4.2889)	**OK**
3	lista 90% ma minimalną długość (bez ostatniego wyrazu < 90%) (11148 wyrazów)	**OK**
3	liczności wyrazów na liście 90% zgodne z korpusem	**OK**
3	tabela Zipfa obejmuje wszystkie wyrazy, r·f policzone poprawnie	**OK**
3	wykładnik Zipfa w raporcie zgodny z obliczonym (s = 0.9584)	**OK**
3	rdzeń: 100 różnych wyrazów	**OK**
3	rdzeń: liczby sąsiadów, krawędzi wychodzących i wchodzących poprawne	**OK**
3	rdzeń to faktycznie 100 wyrazów o największej liczbie sąsiadów (próg: 129 sąsiadów)	**OK**
4	wybrano 100 różnych czasowników i 50 różnych rzeczowników	**OK**
4	każdy czasownik ma dopełnienie w ≥ 20% wystąpień (jest przechodni)	**OK**
4	zbiór treningowy: 50 wektorów po 100 składowych	**OK**
4	kolumny zbioru treningowego = wybrane czasowniki	**OK**
4	suma wektora = liczba wystąpień rzeczownika jako dopełnienia	**OK**
4	wybrane składowe wektorów zgodne z korpusem sparsowanym (bez współrzędnych) (донети+одлука: 21, освојити+медаља: 22, прогласити+независност: 10, водити+рат: 3)	**OK**
4	sieć Kohonena: każdy rzeczownik ma neuron na siatce 10×10	**OK**
4	sieć Kohonena: błąd topograficzny = 0 (sąsiedztwo zachowane)	**OK**

Tabela 18. Wyniki testów: zaliczone 35 z 35.

6. Podsumowanie

Realizacja wymagań

Wymaganie	Jak zrealizowano	Wynik	Rozdz.
Wybrać język, zostawić sam korpus	pobranie z Wikipedii, zostawienie tylko 30 liter cyrylicy	149 373 wyrazów, 0 obcych znaków	1
Usunąć znaki specjalne, interpunkcję, liczby, znaki spoza alfabetu	wyrażenie regularne [^абв…ш] → spacja; raport usuniętych znaków; testy	tylko litery i spacje	1, 2, 5
Tabela udziału procentowego liter (prawdopodobieństwa)	Counter + p = n / N	tabela 30 liter	2.2
Entropia tekstu	$H = -\Sigma\ p \log_2 p$	H = 4,3024 bit/znak	2.4
Ciągi kodowe Huffmana i porównanie z entropią	algorytm Huffmana (heapq)	L = 4,3368 bit, H/L = 99,2%	2.3–2.4
Wyrazy pokrywające 90% tekstu	sortowanie i sumowanie udziałów	11 148 wyrazów	3.1
Prawo Zipfa	regresja w skali log-log	s = 0,958	3.2
Graf wyrazów sąsiadujących, krawędzie wychodzące	zbiory sąsiadów, networkx	krawędzie wych./wch./sąsiedzi	3.3
Rdzeń: 100 wyrazów o największej liczbie sąsiadów	ranking stopni węzłów	tabela 100 wyrazów + rysunek	3.3
50 rzeczowników i 100 czasowników przechodnich	analiza składniowa (stanza), relacja obj	tabele czasowników i rzeczowników	4.1–4.2
Wektor o 100 składowych – zbiór treningowy	kolokacje czasownik–dopełnienie	macierz 50 × 100	4.3
Sieć Kohonena	własna implementacja SOM 10 × 10	błąd topograficzny 0%	4.4–4.5
Przedstawienie w przestrzeni 100D czasowników	mapa z czasownikami dominującymi, płaszczyzny składowych, PCA	rysunki map	4.5–4.6
Wpływ wyboru czasowników, 100 najczęstszych czasowników	warianty A, B, C	korelacja A–B = 0,70	4.7
Sprawozdanie PDF, kod na GitHub	generuj_sprawozdanie.py, repozytorium git	ten dokument + README	–

Tabela 19. Lista wymagań z laboratoriów i miejsce ich realizacji.

Najważniejsze wnioski

- **Litery:** rozkład liter jest nierównomierny, dlatego kod Huffmana (4,34 bit/znak) jest o 13% krótszy od kodu 5-bitowego i tylko o 0,034 bitu dłuższy od teoretycznego minimum, czyli entropii (4,30 bit/znak).
- **Wyrazy:** kilkaset najczęstszych wyrazów pokrywa połowę tekstu, ale do 90% potrzeba już 11 148 wyrazów, bo serbski ma bogatą odmianę. Częstości wyrazów spełniają prawo Zipfa (s = 0,96).
- **Graf:** sieć sąsiedztwa wyrazów jest bezskalowa, a jej rdzeń tworzą słowa funkcyjne.
- **Sieć Kohonena:** ucząc się bez nauczyciela, pogrupowała rzeczowniki w sensowne obszary (posiadanie, przestrzeń, nadawanie nazw i tytułów, działania polityczne i wojskowe). Wynik zależy od wyboru czasowników i od definicji kolokacji. Najlepszą mapę daje wariant z czasownikami faktycznie przechodnimi i kolokacjami wyznaczonymi składniowo.

Uruchomienie programu

`pip install -r requirements.txt`, a następnie `python uruchom_wszystko.py` (lub kolejno lab01.py → lab02… → lab03… → lab04_parsowanie.py → lab04_kohonen.py → sprawdz_wyniki.py → generuj_sprawozdanie.py). Pobrany korpus i wynik analizy składniowej są w repozytorium, więc pełne przeliczenie trwa kilka minut.

Bibliografia

- C. E. Shannon, *A Mathematical Theory of Communication*, Bell System Technical Journal 27, 1948.

- D. A. Huffman, *A Method for the Construction of Minimum-Redundancy Codes*, Proceedings of the IRE 40(9), 1952.
- G. K. Zipf, *Human Behavior and the Principle of Least Effort*, Addison-Wesley, 1949.
- T. Kohonen, *Self-Organized Formation of Topologically Correct Feature Maps*, Biological Cybernetics 43, 1982.
- P. Qi, Y. Zhang, Y. Zhang, J. Bolton, C. D. Manning, *Stanza: A Python Natural Language Processing Toolkit for Many Human Languages*, ACL 2020 (System Demonstrations).
- Universal Dependencies – treebank UD_Serbian-SET, https://universaldependencies.org
- MediaWiki API (rozszerzenie TextExtracts), https://www.mediawiki.org/wiki/API:Main_page